

Final Challenge Presentation



Josue Ureña Valencia
A01738940

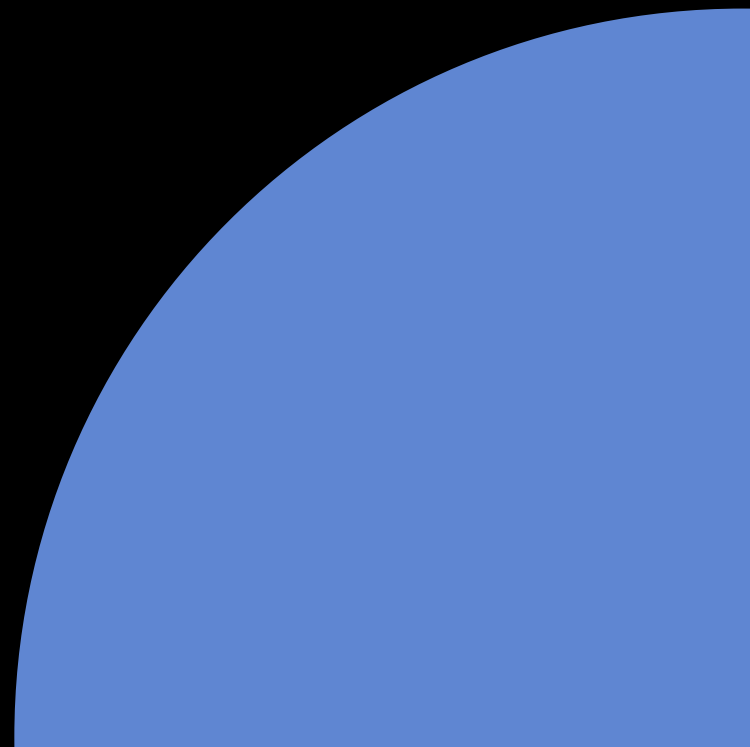
César Arellano Arellano
A00839373



Jose Eduardo Sanchez Martinez
A01738476

Rafael André Gamiz Salazar
A00838280





Introduction

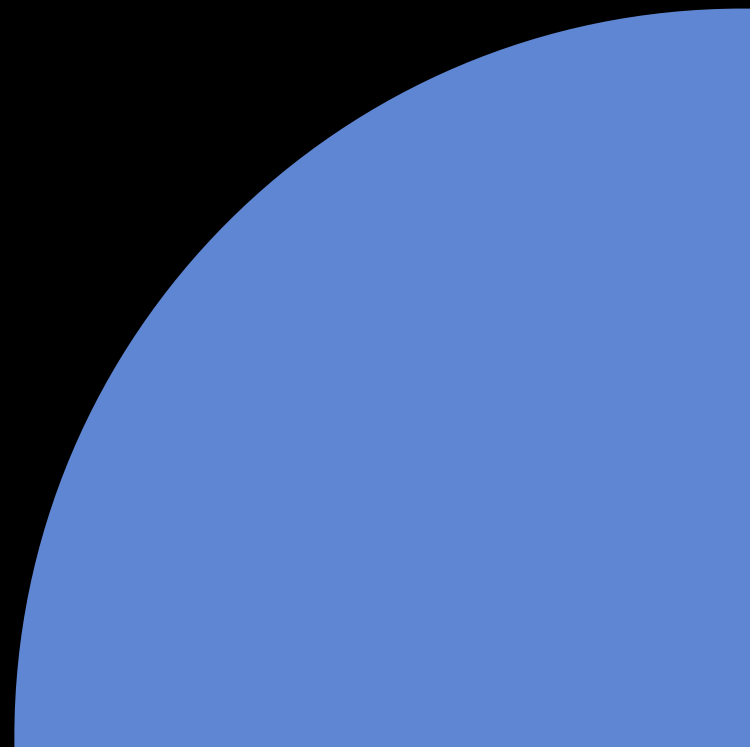


Introduction

Speed control of DC motors is a fundamental problem in robotics, automation, and embedded systems. A motor operating in open loop cannot guarantee that the desired speed will be maintained, since variations in load, voltage, and friction affect its behavior. For this reason, closed-loop control strategies are commonly used to regulate motor speed accurately.

In this final challenge, the objective is to design and implement a closed-loop speed control system for a DC motor. The system must regulate the motor speed according to a reference signal generated from ROS, while estimating the actual motor velocity through encoder feedback. To achieve this, the solution integrates hardware and software components, including an ESP32 microcontroller, an L298N motor driver, a DC motor with encoder, and ROS nodes for command input and monitoring.



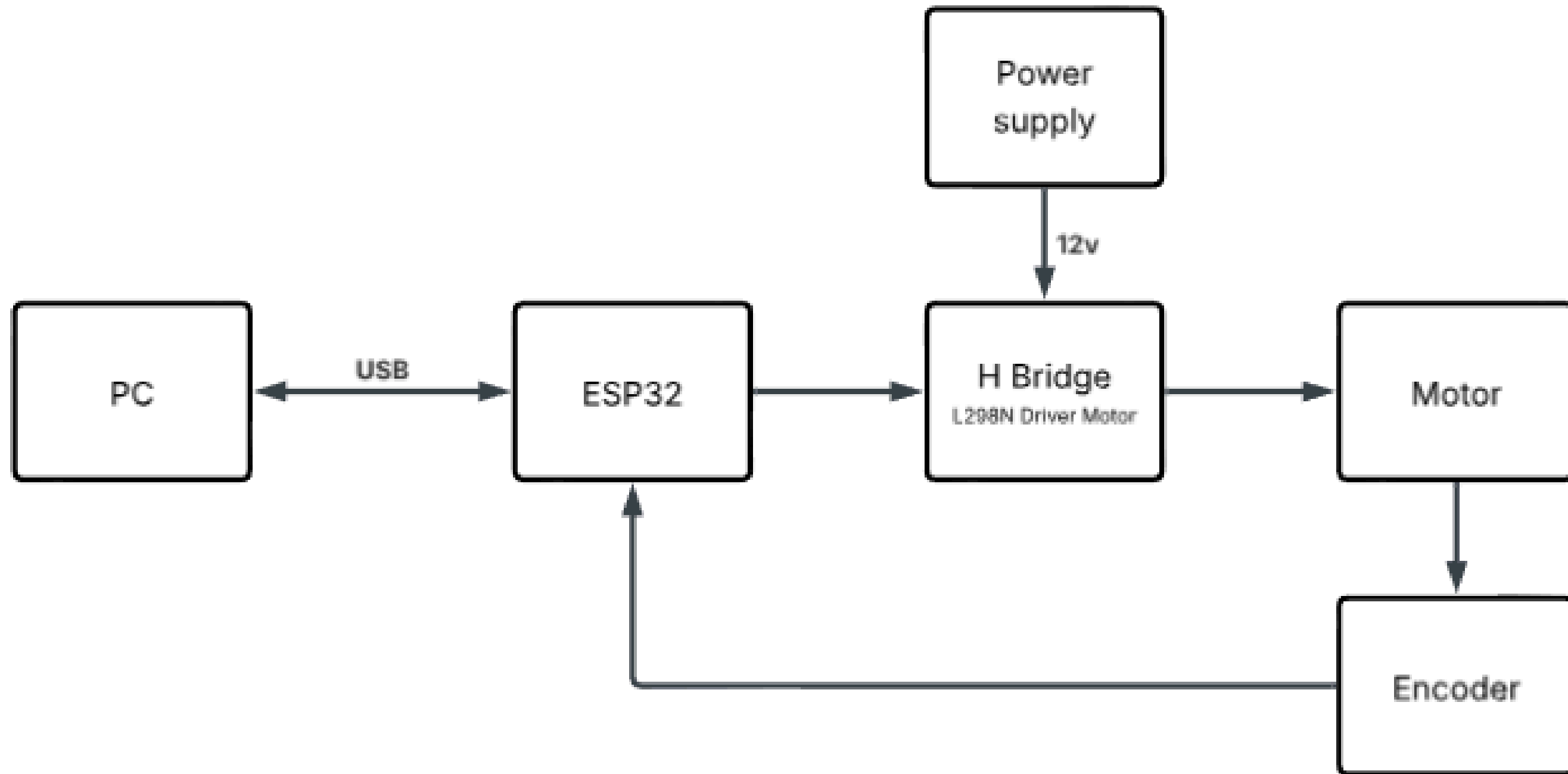


Methodology

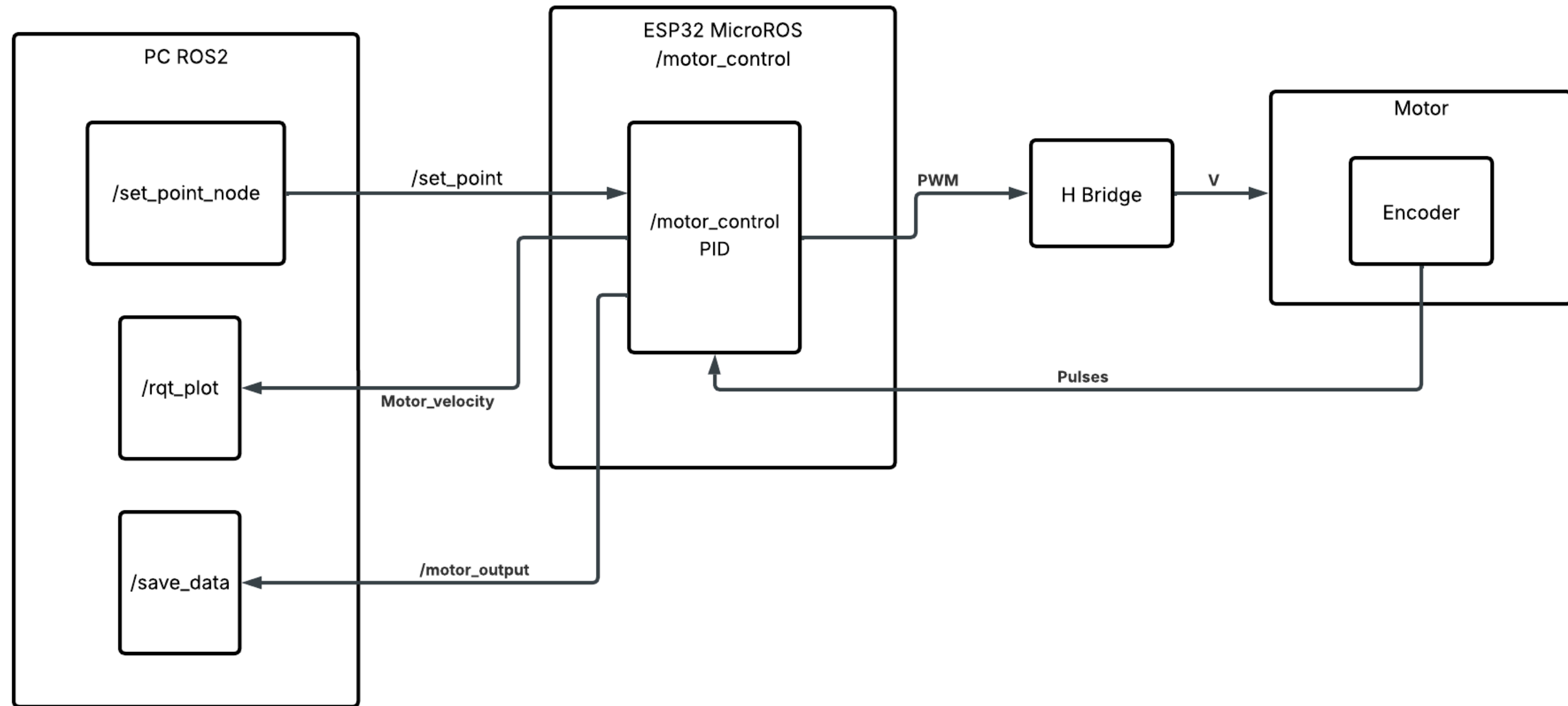
Materials and Components Required for Implementation

- ESP32 Microcontroller – Used to implement the control algorithm and manage communication with the ROS system.
- Brushed DC Motor with Encoder (JGA25-370) – The plant of the system, providing rotational motion and feedback for speed estimation.
- L298N Motor Driver – Power interface used to control the motor direction and regulate the applied voltage through PWM.
- Power Supply – Provides the required voltage and current to operate the motor and electronic components.
- Connecting Wires and Protoboard – Used to establish electrical connections between the microcontroller, motor driver, and sensors.
- Computer running ROS – Generates reference signals, receives system data, and enables communication with the microcontroller.

Block Diagram of the Hardware Components



Block Diagram of the Software Components

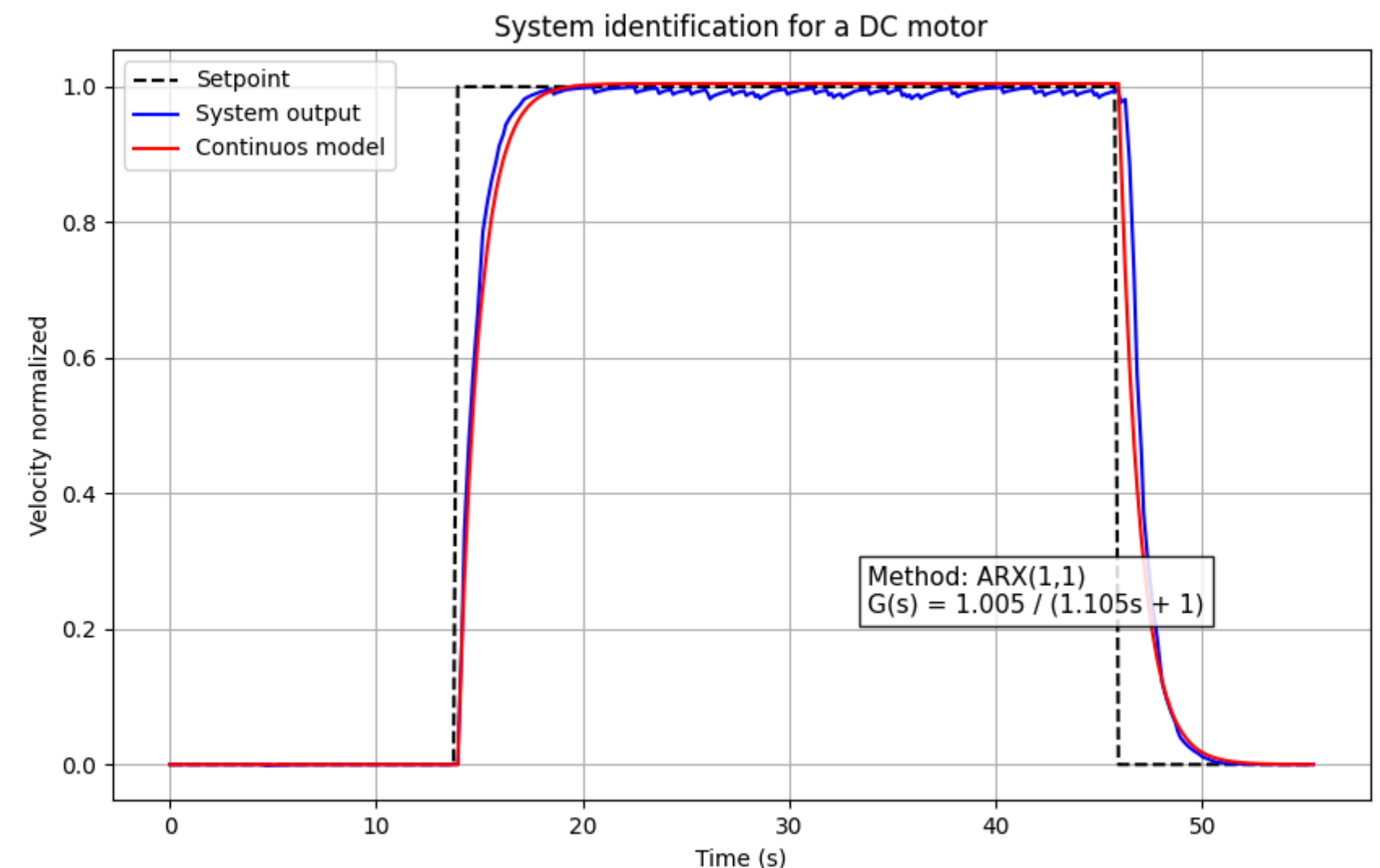


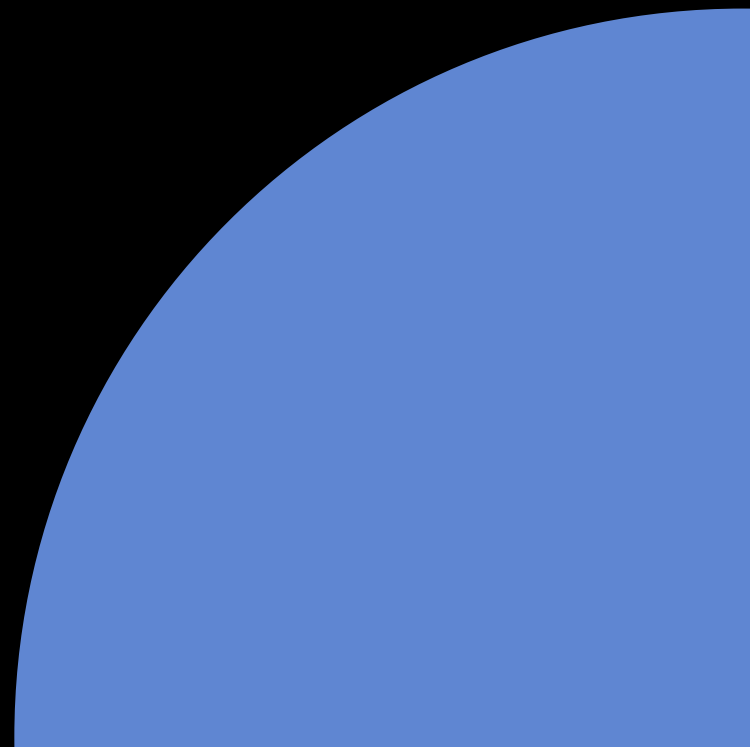
Implemented Control Strategy

First, we identified the motor dynamics from its step response. Using an ARX approximation, we obtained a first-order model described by $G(s) = 1.005 / (1.105s + 1)$. Based on this model, we initially estimated the controller gains using the Ziegler–Nichols tuning method. However, the response obtained with those gains did not provide the desired performance.

For this reason, we performed an additional manual tuning process, adjusting the gains experimentally until we achieved a more stable and accurate response.

The final gains used in the implementation were $K_p = 1.6$, $K_d = 0.02$, and $K_i = 0.6$. The resulting control signal is then saturated and converted into a PWM signal to drive the motor through the motor driver.

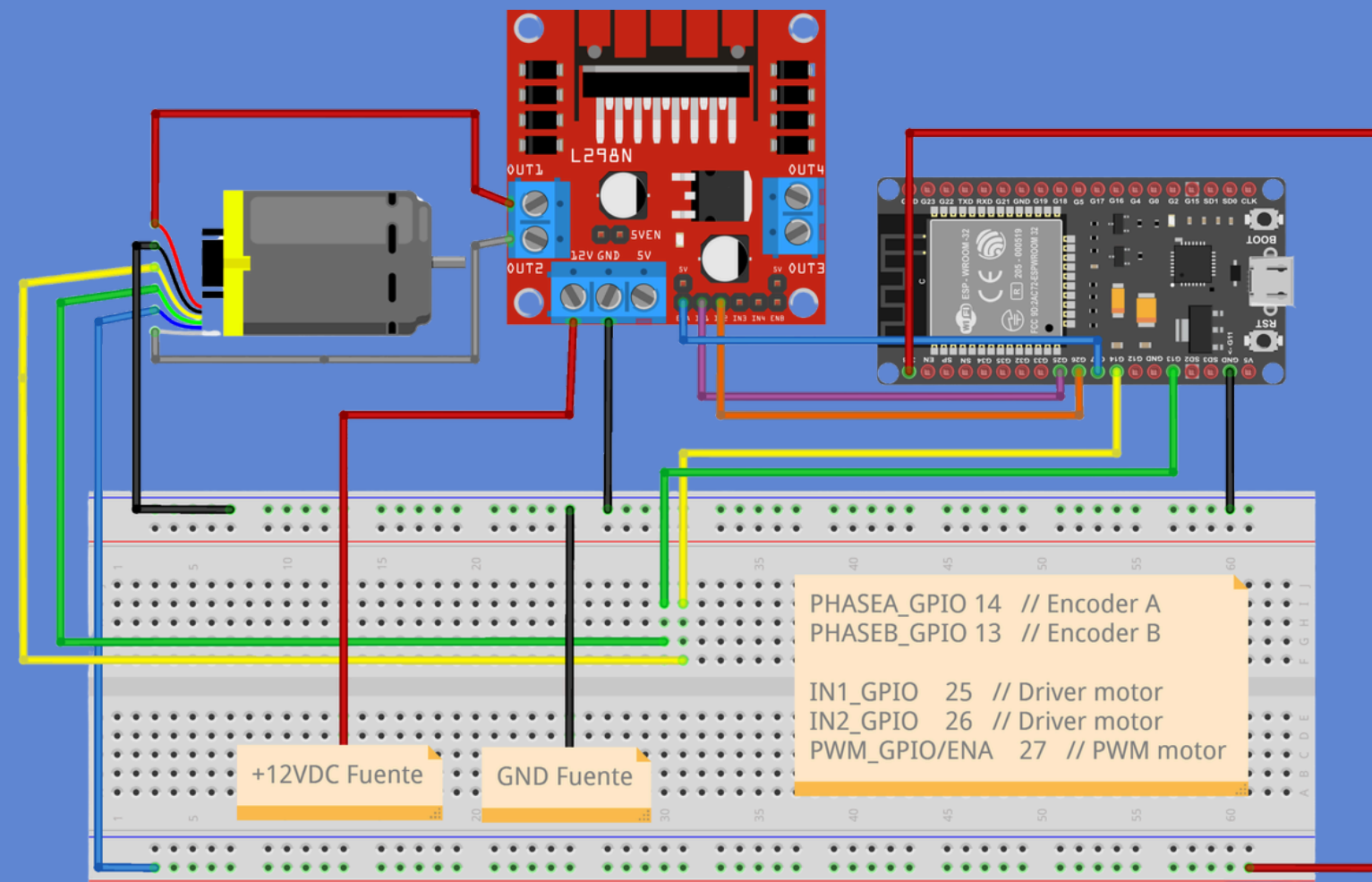




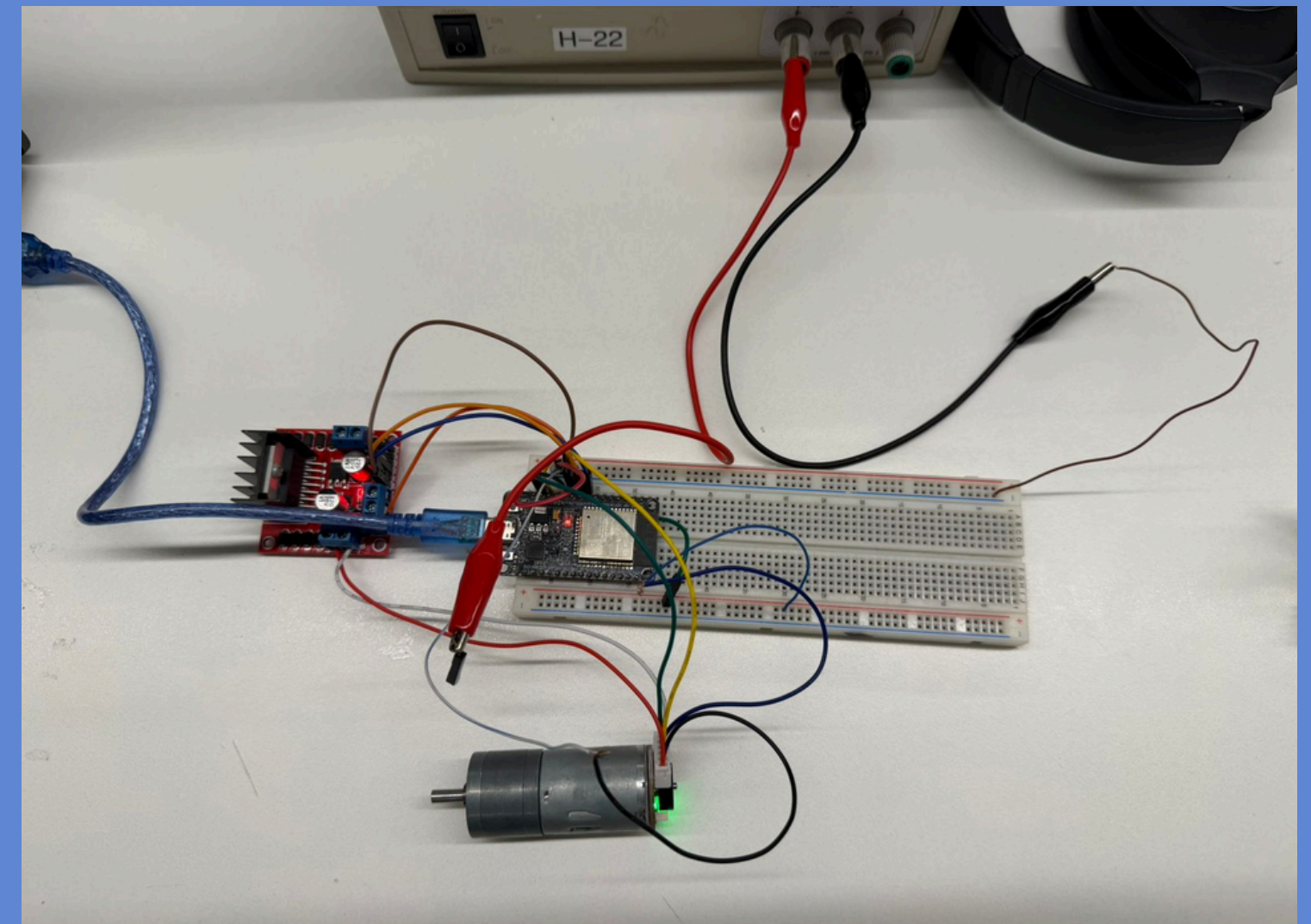
Implementation

Hardware Connections

Real



Connection diagram



Explanation of the Microcontroller Code

Incremental PID

The incremental PID only computes the change in control signal each cycle, preventing integrator windup and avoiding sudden jumps on startup.

```
//PID incremental
controlSignal = controlSignal
+ Kp * (e0 - e1)
+ Ki * Ts * e0
+ (Kd / Ts) * (e0 - 2.0 * e1 + e2);

//SATURATION
controlSignal = constrain(controlSignal,0,1);
```

Encoder reading

The motor speed measurement is performed using an interrupt associated with the encoder.

```
// ISR Encoder
int dir = 1;
void IRAM_ATTR isr() {
    if(digitalRead(PHASEB_GPIO)){
        dir = 1;
    }else{
        dir = -1;
    }countPhaseA++;
}
```



Speed calculation

The motor speed is calculated using the number of pulses detected during a sampling period. A filter was applied and it was normalized to give values between -1 and 1.

```
//RPM
rpm_raw = (pulseCount * 60.0) / (PULSES_PER_REV * Ts);
rpm_raw = clampf(rpm_raw,0.0,RPM_MAX * 2.0);
rpm_filt = alphaRPM * rpm_raw + (1.0 -alphaRPM) * rpm_filt;
rpm = rpm_filt/RPM_MAX;
```

PWM signal generation

The control signal obtained from the PID is converted into a PWM duty cycle that regulates the motor speed. The direction of rotation was determined by the sign of the set point

```
uint32_t duty = (uint32_t)(controlSignal * PWM_MAX);

// Controlar dirección
if(setPoint_RPM > 0){
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
} else if(setPoint_RPM < 0){
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
} else{
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, HIGH);
    duty = 0;
}
ledcWrite(PWM_CHNL, duty);
```



Communication with ROS

The microcontroller creates a ROS node, then a subscriber is created to read the set point. Two publishers are created: one packages time, set point, control signal, and motor speed for later analysis. The other only publishes the motor speed for analysis in an RQT plot.

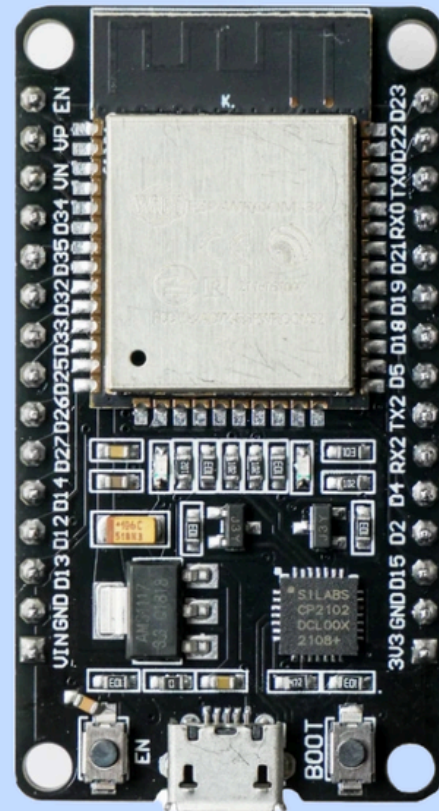
```
// create node
RCCHECK(rclc_node_init_default(&node, "motor_control", "", &support));

// create subscriber
RCCHECK(rclc_subscription_init_default(
    &subscriber,
    &node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Float32),
    "set_point"));

RCCHECK(rclc_publisher_init_default(
    &publisher,
    &node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Float32MultiArray),
    "motor_output"));
RCCHECK(rclc_publisher_init_default(
    &publisher_vel,
    &node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Float32),
    "motor_velocity"));
```

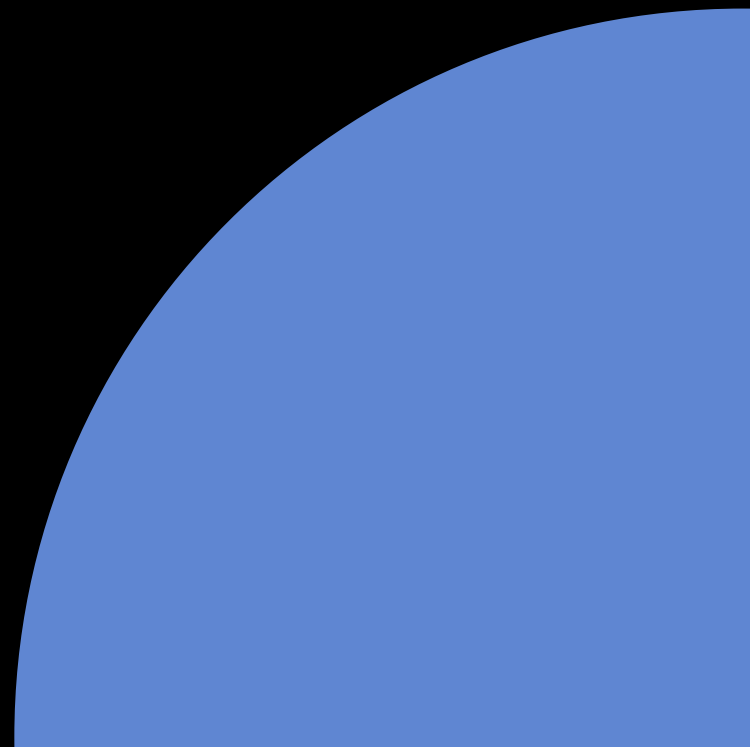
Communication Protocol

ROS2



The communication between the ESP32 and ROS2 is implemented using micro-ROS over a serial connection. The ESP32 works as a ROS node called `motor_control`, which receives the speed reference from the topic `set_point` and publishes the measured motor speed to the topic `motor_output`.

On the ROS side, a separate node generates reference signals such as step, sine, square, or triangle waves and continuously publishes them to `set_point`. The ESP32 processes this reference, executes the PID controller, drives the motor through PWM, and sends the resulting speed back to ROS for monitoring.



Results

Experiments Performed

To evaluate the performance of the proposed control system, four experimental tests were conducted using different reference signals generated in ROS.

1. Sine Wave Reference

A sinusoidal reference signal was applied to evaluate the controller's ability to track continuously varying speed commands.

2. Square Wave Reference

A square wave signal was used to analyze the system response to rapid changes between two speed levels.

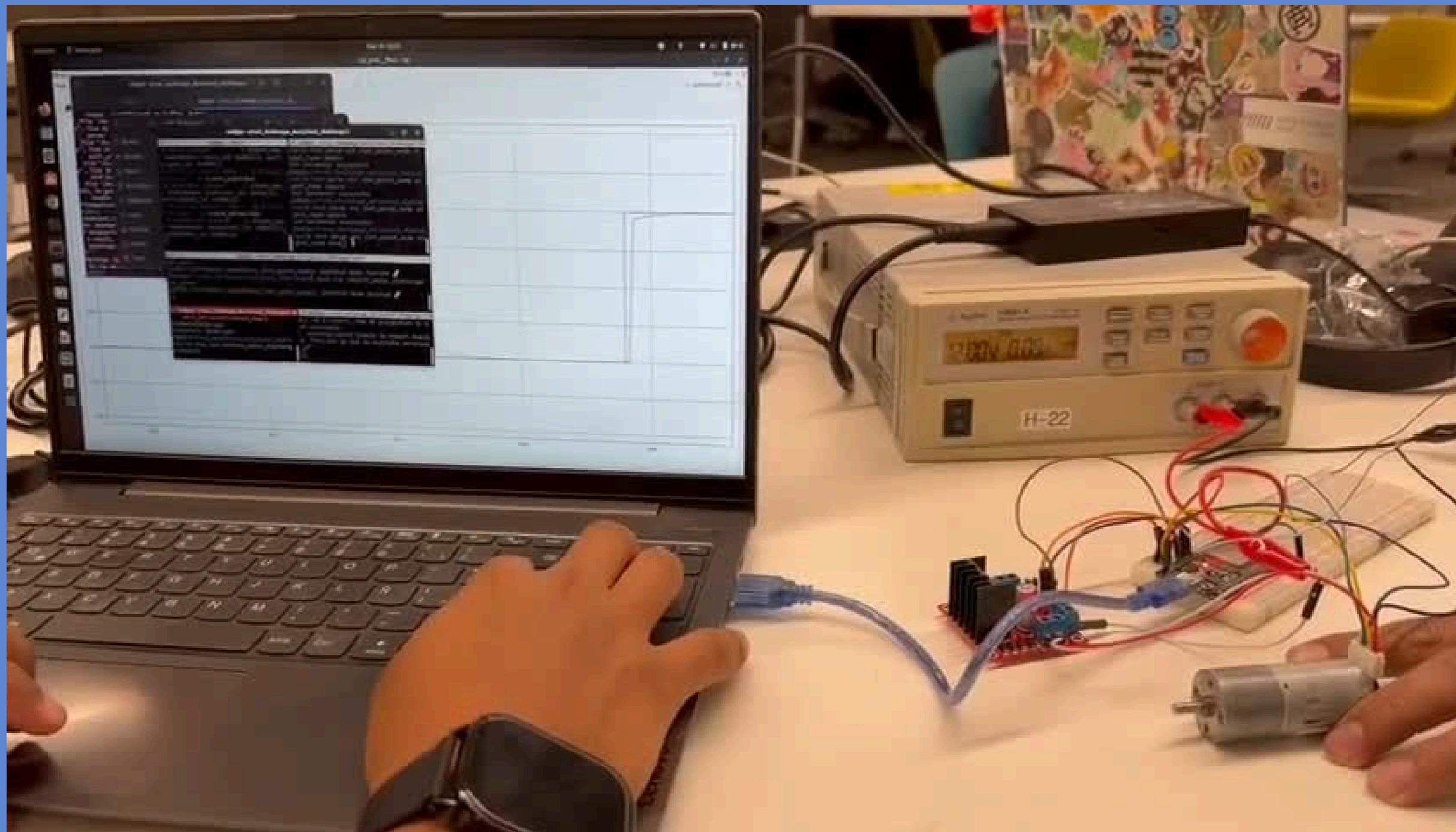
3. Step Reference

A step input was applied to evaluate the transient response of the controller and its ability to reach and stabilize at the desired speed.

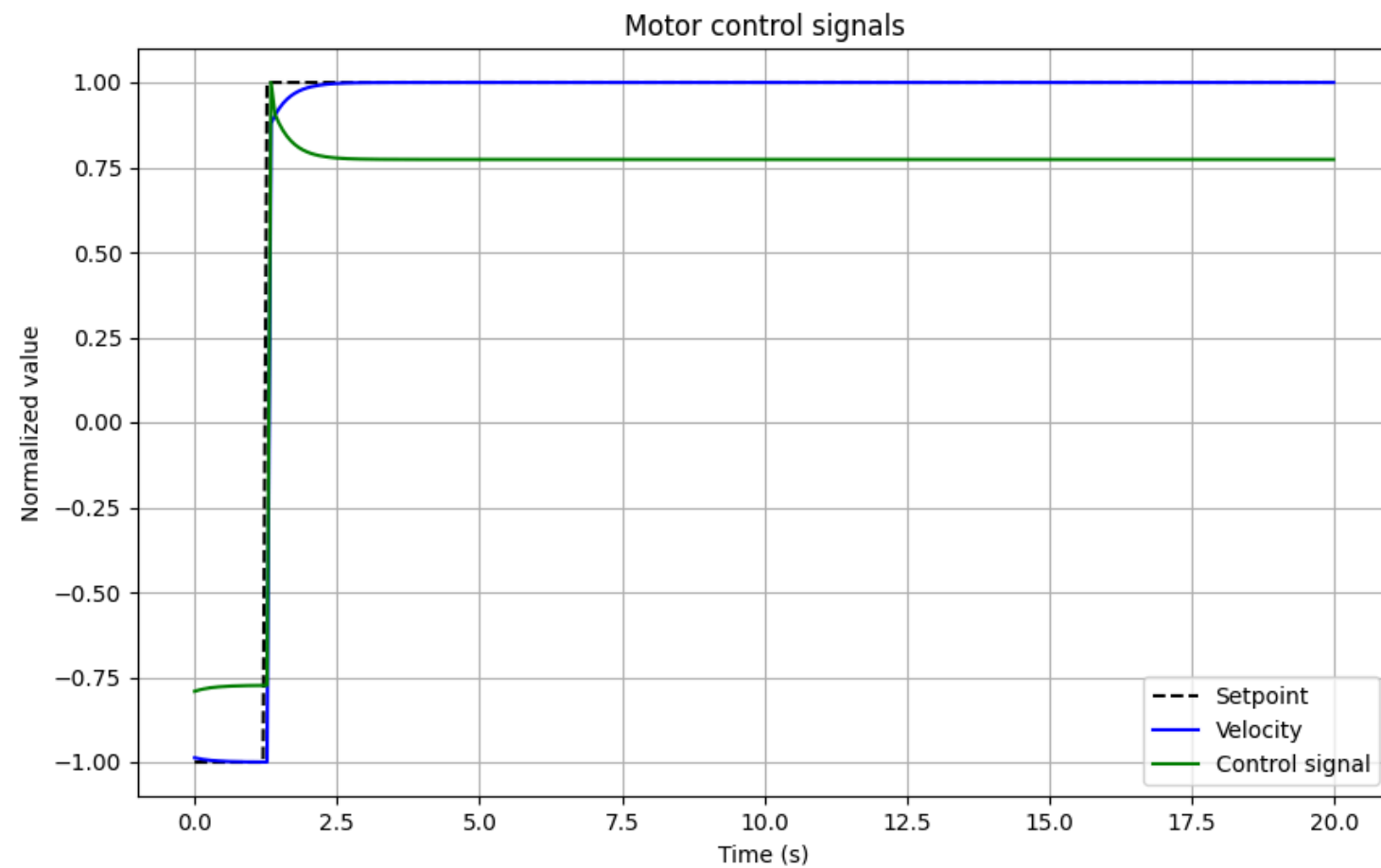
4. Step Reference with Disturbances

A step signal was applied while external disturbances were introduced by manually holding the motor shaft. This test was performed to evaluate the robustness of the controller and its ability to recover the desired speed.

Demonstration

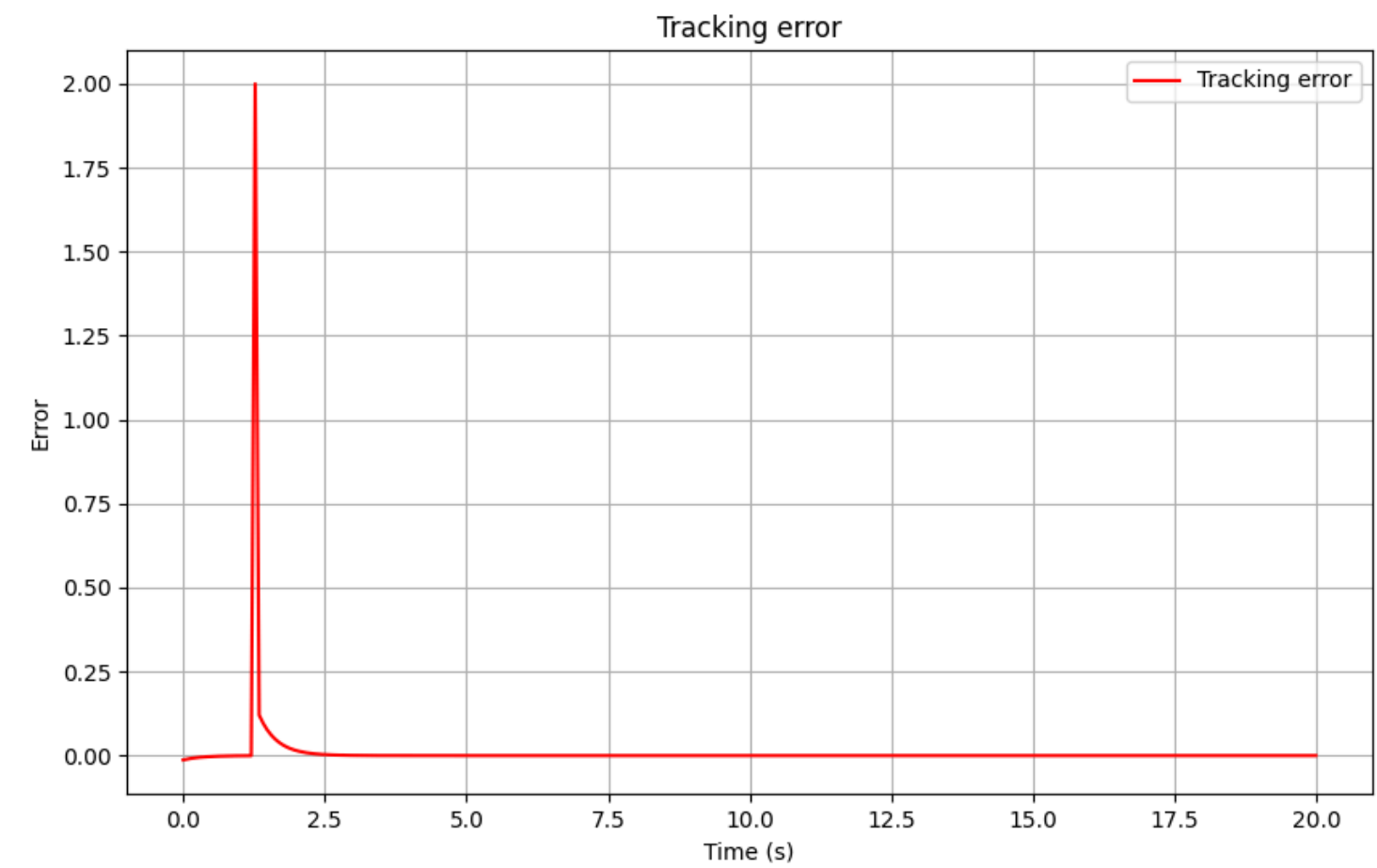


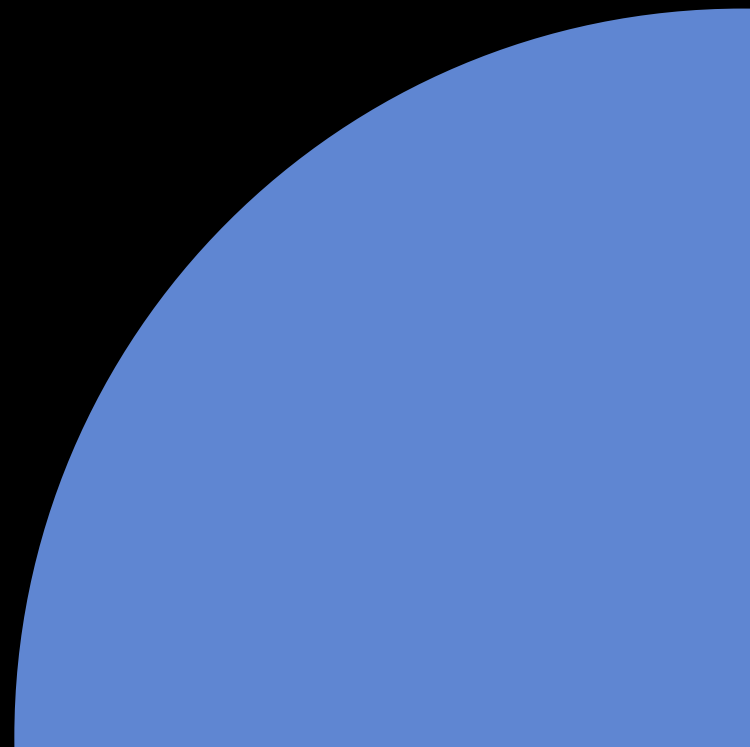
Step Graphs



Motor Control Signals

Tracking Error Control Signals





Conclusion

Conclusion

Based on the step response and the metrics obtained, it can be concluded that the implemented controller exhibits good overall performance. The system shows a fast response, with a rise time of 0.07 s, and no overshoot, indicating stable behavior. Furthermore, the settling time of 1.97 s shows that the system stabilizes within a relatively short period.

In addition, the steady-state error is practically zero, demonstrating that the controller is capable of tracking the reference with high accuracy once steady state is reached. Taken together, these results indicate that the controller design and implementation adequately meet the control objectives set for the system.

Control Performance	
Peak Value	1.00
Peak Time (s)	5.43
Rise Time (s)	0.07
Overshoot (%)	0.00
Settling Time (s)	1.97
Steady-State Error	0.0000
RMSE	0.1191
Max Error	1.9998